

Chapter 3 — Practical Task 01: Hash Index on an Append-Only Log

Road to Architect — Designing Data-Intensive Applications

Chapter 3: Storage and Retrieval

Where this comes from

“Let’s say our data storage consists only of appending to a file ... Then the simplest possible indexing strategy is this: keep an in-memory hash map where every key is mapped to a byte offset in the data file.” — Kleppmann, *DDIA*, Chapter 3

What you already have right

- Writes to `simplest_db.sh` are $O(1)$: `>>` just appends. Reads are $O(n)$: `grep` scans everything.
- An index is **extra data derived from the primary data**. It speeds up reads and costs something on every write. *Every* index makes that trade.
- Hash index: `key` → **byte offset** of the key’s latest record, not the value itself.

One precision point: it has to be a **byte offset**, not a line number. With a line number you’d still have to scan from the start to find line 48,213, because lines aren’t fixed-width. A byte offset lets you `seek()` straight to the record.

Part 1 — Build it

`practical_01_hash_index/` (next to this PDF) is ready:

- `hash_db.py` uses the same `key,value\n` format as your `database` file. File handles, `encode/decode`, and the baseline `scan_get` (your `db_get`, in Python) are already filled in.
- **Your part:** three TODOs, `set`, `get`, and `rebuild_index`. Together they’re about 12 lines.
- `bench.py` writes `N` records, checks your index against the scan, then times both, simulates a restart, and measures index RAM. Don’t edit it.

```
cd chapter3/practical_01_hash_index
python3 bench.py                # 200k writes over 20k keys
```

You’re done when it prints `correctness: OK`. If it doesn’t, the `assert` names the key where your index and the scan disagree.

Part 2 — Predict, then run

Write down your guesses **before** running each command.

Run	Question	Your guess
<code>python3 bench.py</code>	How many times faster is <code>get</code> than <code>scan_get</code> ?	
same	The file is ~9.5 MB, but how many of its 200k lines are still live?	
<code>python3 bench.py 1000000</code>	Will the index use more or less RAM than the file uses on disk?	
same	How long does a restart (<code>rebuild on boot</code>) take? Scale that to a 500 GB log.	

Part 3 — Make the call

For each scenario, choose **no index** or **hash index**, or write **neither, need something else**. Give the one constraint that decided it.

1. **Video play counter.** Around 50k videos. Every play does `set(video_id, count+1)`. The dashboard reads counts constantly.
2. **Audit log.** 10k events/sec written. It's read about once a quarter, when compliance asks "what did user X do in March?"
3. **Orders by date.** Keys look like `order:2026-01-14:8812`. The main query is "all orders between Jan 1 and Jan 31".
4. **User sessions.** 2 billion distinct session keys, on a box with 16 GB RAM. Use your RAM-per-entry from Part 2.
5. **Restart SLA.** Same store as #1, but the log has grown to 500 GB and ops needs the node back within 30 seconds after a crash.

Reveal

Part 2 (measured on an M-series Mac; yours will be close):

- Speedup is about $25,000\times$ at 200k lines and about $70,000\times$ at 1M lines. The scan grows with the file. The index lookup stays flat ($\sim 2\text{--}4\ \mu\text{s}$). That's $O(n)$ vs $O(1)$ you can actually feel.
- Only 20k of the 200k lines are live; **90% of the file is dead**, i.e. overwritten values. The index doesn't grow, but the file does, forever. Kleppmann's next idea, **segments + compaction**, solves exactly this.
- With 1M mostly distinct keys, the index took **$\sim 76\text{ MB}$ of RAM for a $\sim 50\text{ MB}$ file**. That's about 120 bytes per entry in Python: the key string, an int, and the hash-slot overhead. The index is *bigger than the data*. Real engines pack it tighter (Bitcask is about 40 bytes + key), but the shape is the same: **RAM scales with the number of distinct keys**.
- Rebuilding took $\sim 2.5\text{ s}$ for 50 MB, which works out to **$\sim 7\text{ hours}$ for 500 GB**. The index only exists in memory, so every restart pays this.

Part 3:

#	Call	Deciding constraint
1	Hash index	Few keys (fits in RAM easily), many overwrites, point reads. This is Bitcask's textbook sweet spot.
2	No index	Reads are rare and tolerate slowness. An index would cost RAM forever and slow every one of the 10k writes/sec just to speed up a quarterly query. Scan (or batch-process) when asked.
3	Neither, need something else	A hash scatters keys, so neighbours aren't near each other. A range query has to check <i>every</i> key. You need keys kept sorted (SSTables / LSM-trees, B-trees: the next sections).
4	Neither, need something else	$2B \times \sim 40\text{--}120\text{ B}$ is 80–240 GB, which won't fit in 16 GB. Options: shard across machines (Chapter 6), or use an index that lives on disk (B-tree / LSM).
5	Hash index + snapshot	Full-scan rebuild is about 7 hours. Fix: periodically write the index itself to disk (Bitcask calls this a hint file), load it on boot, and replay only the log tail written after it.

When to use a hash index vs the alternatives

Use a hash index when: reads are point lookups by exact key, all the distinct keys fit in RAM with headroom, and the workload is update-heavy on a bounded key set.

Don't, when:

- you need **range queries** or sorted iteration $\rightarrow$ sorted index (LSM / B-tree)
- **distinct keys outgrow RAM** $\rightarrow$ on-disk index, or shard
- **reads are rare** $\rightarrow$ skip the index and scan; don't tax every write for them
- **restart time matters** and the log is large $\rightarrow$ you need snapshots/hint files, which add complexity

The general lesson, true of every index: you pay on every write and in memory/disk so that some *specific* query shape gets faster. Choose an index by asking “which query am I buying speed for, and can I afford the write

and RAM bill?”

Side topic: why values don't need to fit in RAM

“The values can use more space than there is available memory, since they can be loaded from disk with just one disk seek. If that part of the data file is already in the filesystem cache, a read doesn't require any disk I/O at all.” — Kleppmann, *DDIA*, Chapter 3

That sentence is really three separate claims.

1. Only the keys and offsets have to fit in RAM. The values stay on disk.

RAM (the dict)	DISK (the log file)
-----	-----
"user_7" -> 0	-----> 0: user_7,{ ...2 MB metadata... }
"user_42" -> 2,000,113	-----> 2.0MB: user_42,{ ...5 MB blob... }
"user_9" -> 7,000,560	-----> 7.0MB: user_9,{ ... }

Each entry in the dict is a short key plus one integer, and it's the same size whether the value is 10 bytes or 10 MB. So you can have a 16 GB machine with a 2 TB data file. What has to fit in RAM is the **number of distinct keys**, not the total size of the data.

2. “Just one disk seek”. The dict lookup gives you an exact position, so a read never has to search. It's one jump plus one read:

```
offset = self.index[key]      # RAM: about a microsecond
self._reader.seek(offset)     # jump to that byte position on disk
self._reader.readline()       # read exactly that record
```

Compare that with `scan_get`, which reads the whole file. The cost no longer grows with the size of the file. A read costs one disk access, whether the file is 1 MB or 1 TB.

3. “Filesystem cache” means no disk I/O at all. The OS keeps its own cache of recently read file data, in whatever RAM you aren't using. It's called the **page cache**. Your program doesn't ask for it, it doesn't show up as memory your process uses, and Python isn't involved at all.

```
your process --read()--> kernel: "is this 4 KB block of the file already in RAM?"
                           |-- yes -> copy it from RAM           (~microseconds, no disk)
                           '-- no  -> read it from disk, keep a copy in RAM,
                           then return it                       (~0.1 ms SSD, ~10 ms HDD)
```

So with a hash index, a read is:

- one disk access for **cold** data (not read recently),
- **zero** disk accesses for **hot** data (read recently, so the OS still has it in RAM).

Real workloads have hot keys, like popular videos or active users. Those end up served entirely from RAM without your code doing anything.

See it yourself (run `bench.py` first so `bench.db` exists):

```
cd chapter3/practical_01_hash_index
sudo purge                # empties the OS page cache (macOS)
time cat bench.db > /dev/null # cold: has to read from disk
time cat bench.db > /dev/null # warm: noticeably faster, served from RAM
```

Why this matters for the design: Bitcask doesn't build its own cache for values. It relies on the OS to keep hot values in memory. Your process holds only the index, and the OS uses whatever RAM is left over as a value cache.